

✓ Welcome to Colab!

Google Colab is available in VS Code!



Try the new [Google Colab extension](#) for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the **Extensions** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the **Select Kernel** button in the top right.
- Click **Colab** and then select your desired runtime, sign in with your Google account, and you're all set!

See more details in our [announcement blog here](#).

✓ Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to the [getting started with google colab ai](#).

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

✓ Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code, and audio.

How to get started?

- Go to [Google AI Studio](#) and log in with your Google account.
- [Create an API key](#).
- Use a quickstart for [Python](#), or call the REST API using [curl](#).

Discover Gemini's advanced capabilities

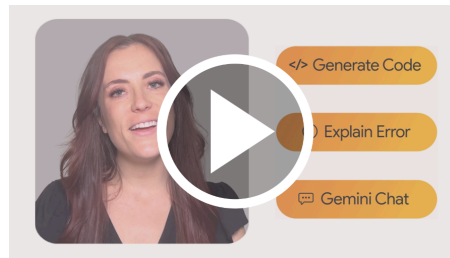
- Play with Gemini [multimodal outputs](#), mixing text and images in an iterative way.
- Discover the [multimodal Live API](#) (demo [here](#)).
- Learn how to [analyze images and detect items in your pictures](#) using Gemini (bonus, there's a [3D version](#) as well!).
- Unlock the power of [Gemini thinking model](#), capable of solving complex task with its inner thoughts.

Explore complex use cases

- Use [Gemini grounding capabilities](#) to create a report on a company based on what the model can find on internet.
- Extract [invoices and form data from PDF](#) in a structured way.
- Create [illustrations based on a whole book](#) using Gemini large context window and Imagen.

To learn more, check out the [Gemini cookbook](#) or visit the [Gemini API documentation](#).

Colab now has AI features powered by [Gemini](#). The video below provides information on how to use these features, whether you're new to Python, or a seasoned veteran.



What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

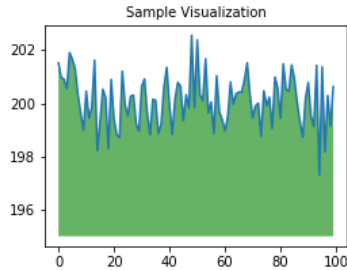
```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
```

```
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Sample Visualization"
display.display(display.Markdown(F"!!![{alt}]({image})"))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10 minute guide](#) or [US stock market data analysis demo](#).

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.

More Resources

Working with Notebooks in Colab

- [Overview of Colab](#)
- [Guide to Markdown](#)
- [Importing libraries and installing dependencies](#)
- [Saving and loading notebooks in GitHub](#)
- [Interactive forms](#)
- [Interactive widgets](#)

Working with Data

- [Loading data: Drive, Sheets, and Google Cloud Storage](#)

- [Charts: visualizing data](#)
- [Getting started with BigQuery](#)

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the [full course website](#) for more.

- [Intro to Pandas DataFrame](#)
- [Intro to RAPIDS cuDF to accelerate pandas](#)
- [Getting Started with cuML's accelerator mode](#)

Using Accelerated Hardware

- [Train a CNN to classify handwritten digits on the MNIST dataset using Flax NNX API](#)
- [Train a Vision Transformer \(ViT\) for image classification with JAX](#)
- [Text classification with a transformer language model using JAX](#)

✓ Featured examples

- [Train a miniGPT language model with JAX AI Stack](#)
- [LoRA/QLoRA finetuning for LLM using Tux](#)
- [Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA](#)
- [Loading Hugging Face Transformers Checkpoints](#)
- [8-bit Integer Quantization in Keras](#)
- [Float8 training and inference with a simple Transformer model](#)
- [Pretraining a Transformer from scratch with KerasHub](#)
- [Simple MNIST convnet](#)
- [Image classification from scratch using Keras 3](#)
- [Image Classification with KerasHub](#)

```
import tensorflow as tf
```

```
# Load the Fashion MNIST dataset
fmnist = tf.keras.datasets.fashion_mnist
```

```
fmnist
```

```
<module 'keras.datasets.fashion_mnist' from '/usr/local/lib/python3.12/dist-packages/keras/datasets/fashion_mnist/__init__.py'>
```

```
# Load the training and test split of the Fashion MNIST dataset
(training_images, training_labels), (test_images, test_labels) = fmnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 0s 0us/step
```

```
import numpy as np
import matplotlib.pyplot as plt

# You can put between 0 to 59999 here
index = 3

# Set number of characters per row when printing
np.set_printoptions(linewidth=320)

# Print the label and image
print(f'\nIMAGE PIXEL ARRAY:\n {training_images[index]}')

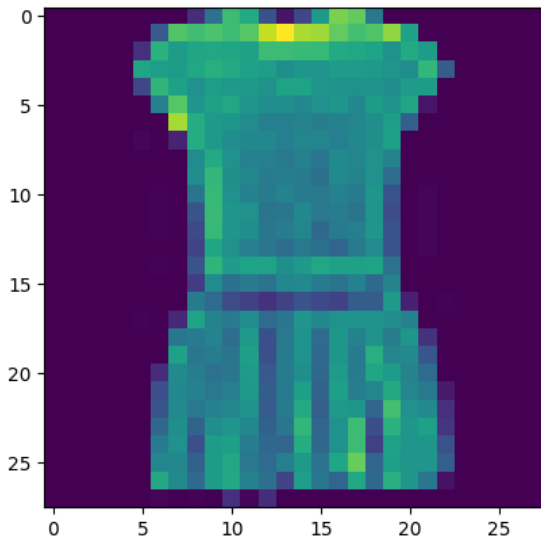
# Visualize the image
```

```
print('LABEL: {training_labels[index]}')
plt.imshow(training_images[index])
plt.show()
```

IMAGE PIXEL ARRAY:

```
[[ 0  0  0  0  0  0  0  0  33  96 175 156  64  14  54 137 204 194 102  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  73 186 177 183 175 188 232 255 223 219 194 179 186 213 146  0  0  0  0  0]
 [ 0  0  0  0  0  35 163 140 150 152 150 146 175 175 173 171 156 152 148 129 156 140  0  0  0  0  0]
 [ 0  0  0  0  0 150 142 140 152 160 156 146 142 127 135 133 140 140 137 133 125 169 75  0  0  0  0]
 [ 0  0  0  0  0  54 167 146 129 142 137 137 131 148 148 133 131 131 131 125 140 140  0  0  0  0  0]
 [ 0  0  0  0  0  0 110 188 133 146 152 133 125 127 119 129 133 119 140 131 150 14  0  0  0  0  0]
 [ 0  0  0  0  0  0  221 158 137 135 123 110 110 114 108 112 117 127 142 77  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  4  0  25 158 137 125 119 119 110 117 117 110 119 127 144  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 123 156 129 112 110 102 112 100 121 117 129 114  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 125 169 127 119 106 108 104 94 121 114 129 91  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  2  0 98 171 129 112 104 114 106 102 112 104 133 64  0  4  0  0  0  0  0]
 [ 0  0  0  0  0  0  2  0 66 173 135 129 98 100 119 102 108 98 135 60  0  4  0  0  0  0  0]
 [ 0  0  0  0  0  0  2  0 56 171 135 127 100 108 117 85 106 110 135 66  0  4  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 52 150 129 110 100 91 102 94 83 104 123 66  0  4  0  0  0  0  0]
 [ 0  0  0  0  0  0  2  0 66 167 140 148 148 127 137 152 146 146 148 96  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 45 123 94 104 96 119 121 106 98 112 87 114  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 106 89 58 50 37 50 66 56 50 75 75 137 22  0  2  0  0  0  0  0]
 [ 0  0  0  0  0  2  0 29 148 114 106 125 89 100 133 117 131 131 131 125 112  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0 100 106 114 91 137 62 102 131 89 135 112 131 108 135 37  0  0  0  0  0]
 [ 0  0  0  0  0  0  0 146 100 108 98 144 62 106 131 87 133 104 160 117 121 68  0  0  0  0  0]
 [ 0  0  0  0  0  0 33 121 108 96 100 140 71 106 127 85 140 104 150 140 114 89  0  0  0  0  0]
 [ 0  0  0  0  0  0 62 119 112 102 110 137 75 106 144 81 144 108 117 154 117 104 18  0  0  0  0]
 [ 0  0  0  0  0  0 66 121 102 112 117 131 73 104 156 77 137 135 83 179 129 121 35  0  0  0  0]
 [ 0  0  0  0  0  0 85 127 81 125 133 119 79 100 169 83 129 175 60 163 135 146 39  0  0  0  0]
 [ 0  0  0  0  0  0 106 129 62 140 144 108 85 83 158 85 129 175 48 146 133 135 64  0  0  0  0]
 [ 0  0  0  0  0  0 117 119 79 140 152 102 89 110 137 96 150 196 83 144 135 133 77  0  0  0  0]
 [ 0  0  0  0  0  0 154 121 87 140 154 112 94 52 142 100 83 152 85 160 133 100 12  0  0  0  0]
 [ 0  0  0  0  0  0  4  0  2  0 35  4 33  0  0  0  0  0  0  0  0  0  0  0  0  0]]
```

LABEL: 3



```
# Normalize the pixel values of the train and test images
training_images = training_images / 255.0
test_images = test_images / 255.0
```

```
# Build the classification model
model = tf.keras.models.Sequential([tf.keras.layers.Flatten(),
                                     tf.keras.layers.Dense(128, activation=tf.nn.relu),
                                     tf.keras.layers.Dense(64, activation=tf.nn.relu),
                                     tf.keras.layers.Dense(32, activation=tf.nn.relu),
                                     tf.keras.layers.Dense(10, activation=tf.nn.softmax)])
```

```
model.compile(optimizer = tf.optimizers.Adam(),
              loss = 'sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(training_images, training_labels, epochs=100)
```



```

# Upload your own image and predict the Fashion MNIST class
# Run this cell AFTER training the model.

from google.colab import files
from PIL import Image, ImageOps
import numpy as np
import matplotlib.pyplot as plt

# Fashion MNIST class names
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

# Upload an image file: png, jpg, jpeg, etc.
uploaded = files.upload()

for file_name in uploaded.keys():
    # 1. Open uploaded image
    img = Image.open(file_name).convert("L") # convert to grayscale

    # 2. Resize image to 28 x 28 because Fashion MNIST images are 28x28
    img = img.resize((28, 28))

    # 3. Fashion MNIST usually has a dark background and light object.
    # If your uploaded image has a white background, invert it.
    img = ImageOps.invert(img)

    # 4. Convert image into numpy array and normalize values from 0 to 1
    img_array = np.array(img) / 255.0

    # 5. Add batch dimension: model expects shape (1, 28, 28)
    img_input = np.expand_dims(img_array, axis=0)

    # 6. Predict
    prediction = model.predict(img_input)
    predicted_index = np.argmax(prediction[0])
    confidence = np.max(prediction[0]) * 100

    # 7. Show result
    plt.imshow(img_array, cmap="gray")
    plt.axis("off")
    plt.title(f"Prediction: {class_names[predicted_index]} ({confidence:.2f}% confidence)")
    plt.show()

    print("Uploaded file:", file_name)
    print("Predicted class number:", predicted_index)
    print("Predicted class name:", class_names[predicted_index])
    print(f"Confidence: {confidence:.2f}%")
    print("All class probabilities:")
    for name, prob in zip(class_names, prediction[0]):
        print(f"{name}: {prob*100:.2f}%")

```

Choose Files bag.jpg

bag.jpg(image/jpeg) - 20096 bytes, last modified: 7/13/2026 - 100% done

Saving bag.jpg to bag.jpg

1/1 ————— 0s 40ms/step

Prediction: Bag (100.00% confidence)



Uploaded file: bag.jpg

Predicted class number: 8

Predicted class name: Bag

Confidence: 100.00%

All class probabilities:

T-shirt/top: 0.00%

Trouser: 0.00%

Pullover: 0.00%

Dress: 0.00%

Coat: 0.00%

Sandal: 0.00%

Shirt: 0.00%

Sneaker: 0.00%

Bag: 100.00%

Ankle boot: 0.00%

```

# Install Gradio
!pip install gradio -q

import gradio as gr
import numpy as np
from PIL import Image, ImageOps
import matplotlib.pyplot as plt

# Fashion MNIST class names
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

def predict_fashion_image(uploaded_image):
    """
    This function takes an uploaded image,
    preprocesses it like Fashion MNIST,
    and predicts its class using the trained model.
    """

    # Convert uploaded image to grayscale
    img = uploaded_image.convert("L")

    # Resize image to 28x28 because Fashion MNIST images are 28x28
    img = img.resize((28, 28))

    # Invert image colors to match Fashion MNIST style
    img = ImageOps.invert(img)

    # Convert image to numpy array
    img_array = np.array(img)

    # Normalize pixel values from 0-255 to 0-1
    img_array = img_array / 255.0

```



```
# Add batch dimension: (28, 28) becomes (1, 28, 28)
img_input = np.expand_dims(img_array, axis=0)

# Predict using trained model
prediction = model.predict(img_input)

# Get predicted class index
predicted_index = np.argmax(prediction[0])

# Get confidence percentage
confidence = np.max(prediction[0]) * 100

# Create result text
result = f"Prediction: {class_names[predicted_index]}\nConfidence: {confidence:.2f}%"

# Return all class probabilities also
probabilities = {
    class_names[i]: float(prediction[0][i])
    for i in range(len(class_names))
}

return result, probabilities

# Create Gradio interface
demo = gr.Interface(
    fn=predict_fashion_image,
    inputs=gr.Image(type="pil", label="Upload Fashion Image"),
    outputs=[
        gr.Textbox(label="Prediction Result"),
        gr.Label(num_top_classes=10, label="Class Probabilities")
    ],
    title="Fashion MNIST Image Classifier",
    description="Upload an image of clothing, shoes, or bag. The model will predict the Fashion MNIST class."
)

# Launch app
demo.launch()
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True`.

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://287ce846a6583f7ee7.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `g

Fashion MNIST Image Classifier

Upload an image of clothing, shoes, or bag. The model will predict the Fashion MNIST class.

Upload Fashion Image



Prediction Result

Prediction: Bag
Confidence: 100.00%

Class Probabilities

Bag